

提炼神经网络中的知识

杰弗里·辛顿^{*}

谷歌公司 山景城

geoffhinton@google.com

Oriol Vinyals

谷歌公司 山景城

vinyals@google.com

杰夫·迪芬

谷歌公司 山景城

jeff@google.com

摘要

提高几乎所有机器学习算法性能的一个非常简单的方法是，在相同数据上训练许多不同的模型，然后对它们的预测结果取平均值 [3]。然而，使用一整个模型集成来进行预测既麻烦又可能计算成本过高，难以部署给大量用户，尤其是当各个模型都是大型神经网络时。Caruana 及其合作者 [1] 已经证明，有可能将集成模型中的知识压缩到单个模型中，这样的单个模型更易于部署，而我们则使用不同的压缩技术进一步发展了这种方法。我们在 MNIST 数据集上取得了一些令人惊讶的结果，并且表明，通过将集成模型中的知识提炼到单个模型中，我们可以显著改进一个被大量使用的商业系统的声学模型。我们还引入了一种新型集成模型，它由一个或多个完整模型以及许多专家模型组成，这些专家模型负责学习区分那些完整模型容易混淆的细粒度类别。与混合专家模型不同，这些专家模型可以快速且并行地进行训练。

1 引言

许多昆虫的幼虫形态经过优化，能够从环境中摄取能量和营养，而成虫形态则截然不同，其优化方向是满足移动和繁殖这些截然不同的需求。在大规模机器学习中，尽管训练阶段和部署阶段的需求差异很大，我们通常却使用极为相似的模型：对于语音识别和目标识别等任务，训练必须从规模庞大、高度冗余的数据集中提取结构，但无需实时运行，还可以使用大量计算资源。然而，向大量用户部署模型时，对延迟和计算资源的要求要严格得多。

昆虫的类比表明，如果复杂的模型能更轻松地从数据中提取结构，我们应该愿意去训练这类模型。这种复杂模型可以是多个单独训练的模型组成的集成，也可以是一个经过强正则化（如 dropout [9]）训练的大型单模型。一旦复杂模型训练完成，我们就可以采用另一种训练方式——我们称之为“蒸馏”——将知识从复杂模型转移到更适合部署的小型模型中。Rich Caruana 及其合作者 [1] 已经率先实践了这一策略的一种形式。在他们的重要论文中，他们令人信服地证明，大型集成模型所学到的知识可以转移到单个小型模型中。

一种概念上的障碍可能阻碍了对这种极具前景的方法的进一步研究，即我们倾向于将训练好的模型中的知识与学到的参数值等同起来，这使得我们很难理解如何在保持相同知识的情况下改变模型的形式。对知识的一种更抽象的看法——将其从任何特定的实例化中解放出来——是它是一种习得的

^{*}同时隶属于多伦多大学和加拿大高级研究所。

[†]同等贡献。

从输入向量到输出向量的映射。对于那些需要学习区分大量类别的复杂模型而言，常规的训练目标是最大化正确答案的平均对数概率，但学习过程会产生一个副作用——经过训练的模型会为所有错误答案分配概率，即便这些概率非常小，其中一些也会远大于另一些。错误答案的相对概率能让我们深入了解复杂模型的泛化趋势。例如，一张宝马汽车的图片被误判为垃圾车的概率可能极低，但这种误判的可能性仍比将其误判为胡萝卜高出许多倍。

人们普遍认为，用于训练的目标函数应尽可能贴近用户的真实目标。尽管如此，当真正的目标是让模型对新数据有良好的泛化能力时，模型通常还是被训练去优化在训练数据上的表现。显然，训练模型使其具有良好的泛化能力会更好，但这需要知道正确的泛化方式，而这种信息通常是无法获取的。然而，当我们知识从大型模型蒸馏到小型模型中时，我们可以训练小型模型以与大型模型相同的方式进行泛化。例如，如果笨重的大型模型泛化能力强是因为它是大量不同模型集成后的平均值，那么一个被训练成以相同方式泛化的小型模型，在测试数据上的表现通常会远好于以常规方式在训练该集成模型所用的同一训练集上训练出的小型模型。

将复杂模型的泛化能力迁移到小型模型的一个明显方法是，把复杂模型生成的类别概率作为训练小型模型的“软目标”。在这个迁移阶段，我们可以使用相同的训练集，也可以使用单独的“迁移”集。当复杂模型是由多个较简单模型组成的大型集成模型时，我们可以将这些模型各自的预测分布的算术平均值或几何平均值用作软目标。当软目标具有较高的熵值时，它们在每个训练样本中提供的信息比硬目标多得多，而且训练样本之间的梯度方差也小得多，因此小型模型通常可以用比原始复杂模型少得多的数据进行训练，并且可以使用高得多的学习率。

对于像 MNIST 这样的任务，复杂模型几乎总能以极高的置信度给出正确答案，此时关于所学函数的大部分信息都存在于软目标中极小概率的比例关系里。例如，某个版本的数字 2 被判定为 3 的概率可能是 10^{-6} ，被判定为 7 的概率可能是 10^{-9} ，而对于另一个版本的数字 2，这两个概率可能会颠倒过来。这是很有价值的信息，它定义了数据间丰富的相似性结构（也就是说，它能表明哪些 2 看起来像 3，哪些看起来像 7），但在迁移阶段，由于这些概率非常接近零，所以对交叉熵损失函数的影响很小。卡鲁阿纳及其合作者解决这个问题的方法是，使用 logits（最终 softmax 的输入）而非 softmax 输出的概率作为训练小模型的目标，并且最小化复杂模型与小模型所产生的 logits 之间的平方差。我们提出的一种更通用的解决方案称为“知识蒸馏”，具体做法是提高最终 softmax 的温度，直到复杂模型产生一组足够柔和的目标。然后，在训练小模型以匹配这些软目标时，我们使用相同的高温。稍后我们会说明，匹配复杂模型的 logits 实际上是蒸馏的一种特殊情况。

用于训练小模型的迁移数据集可以完全由未标记数据组成 [1]，也可以使用原始训练集。我们发现，使用原始训练集的效果很好，特别是如果我们在目标函数中添加一个小项，鼓励小模型既预测真实目标，又匹配复杂模型提供的软目标。通常情况下，小模型无法完全匹配软目标，而朝着正确答案的方向犯错被证明是有帮助的。

2 知识蒸馏

神经网络通常通过“softmax”输出层生成类别概率，该层通过将为每个类别计算的 logit (z_i) 与其他 logit 进行比较，将其转换为概率 (a_i)。

$$a_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

其中 T 是一个温度参数，通常设置为 1。使用更高的 T 值会在类别上产生更平缓的概率分布。

在最简单的蒸馏形式中，知识通过以下方式转移到蒸馏模型：在迁移集上训练蒸馏模型，并对迁移集中的每个样本使用一个软目标分布，该分布是通过让复杂模型在其 softmax 函数中使用较高温度的。训练蒸馏模型时使用相同的高温，但训练完成后，它会使用温度 1。

当迁移集中全部或部分样本的正确标签已知时，通过训练蒸馏模型使其输出正确标签，这种方法的效果会显著提升。实现这一目标的一种方式是利用正确标签来修改软目标，但我们发现更好的方法是直接对两个不同的目标函数取加权平均。第一个目标函数是与软目标相关的交叉熵，计算该交叉熵时，蒸馏模型的 softmax 函数所使用的高温与从复杂模型生成软目标时使用的高温相同。第二个目标函数是与正确标签相关的交叉熵，计算时使用蒸馏模型 softmax 函数中完全相同的对数几率，但温度设为 1。我们发现，通常情况下，给第二个目标函数赋予相当低的权重能获得最佳结果。

目标函数。由于软目标产生的梯度大小缩放为 $1/T^2$

在同时使用硬目标和软目标时，将它们乘以 T^2 是很重要的。这能确保在调整元参数进行实验时，即使改变蒸馏所用的温度，硬目标和软目标的相对贡献也能大致保持不变。

2.1 匹配 logits 是蒸馏的一种特殊情况

迁移集中的每个样本都会对蒸馏模型的每个对数几率 z_i 产生一个交叉熵梯度 dC/dz_i 。如果复杂模型的对数几率为 v_i ，其产生的软目标概率为 p_i ，且迁移训练是在温度 T 下进行的，那么该梯度可表示为：

$$\frac{\partial C}{\partial z_i} = \frac{1}{T} (q_i - p_i) = \frac{1}{T} \left(\frac{e^{z_i/T}}{\sum_j e^{z_j/T}} - \frac{e^{v_i/T}}{\sum_j e^{v_j/T}} \right) \quad (2)$$

如果温度相对于 logits 的大小较高，我们可以近似：

$$\frac{\partial C}{\partial z_i} \approx \frac{1}{T} \left(\frac{1 + z_i/T}{N + \sum_j z_j/T} - \frac{1 + v_i/T}{N + \sum_j v_j/T} \right) \quad (3)$$

如果我们现在假设，对于每个迁移情况，logits 都已分别进行零均值化处理，那么 $\sum_j z_j = \sum_j v_j = 0$ 式 3 可简化为：

$$\frac{\partial C}{\partial z_i} \approx \frac{1}{N+1} (z_i - v_i)$$

因此，在高温极限下，蒸馏相当于最小化 $1/2(z_i - v_i)^2$ ，前提是对每个迁移情况的 logits 分别进行零均值处理。在较低温度下，蒸馏对那些远低于平均值的负向 logits 的匹配关注程度要低得多。这可能是有利的，因为这些 logits 几乎完全不受用于训练复杂模型的成本函数的约束，因此它们可能非常嘈杂。另一方面，这些高度负向的 logits 可能传达关于复杂模型所获知识的有用信息。哪种影响占主导地位是一个经验性问题。我们发现，当蒸馏模型过小而无法捕获复杂模型中的所有知识时，中等温度效果最佳，这有力地表明忽略大幅负向的 logits 可能是有帮助的。

3 MNIST 上的初步实验

为了了解蒸馏的效果，我们在全部 60,000 个训练样本上训练了一个大型神经网络，该网络有两个隐藏层，每层包含 1200 个整流线性隐藏单元。正如文献 [5] 中所描述的，我们使用 dropout 和权重约束对该网络进行了强正则化。Dropout 可以看作是一种训练指数级大规模模型集成的方法，这些模型共享权重。此外，输入图像还经过了

在任何方向上抖动最多两个像素。该网络的测试错误为 67 个，而一个规模更小、包含两个各有 800 个整流线性隐藏单元的隐藏层且无正则化的网络，测试错误为 146 个。但如果仅通过增加一项额外任务（即在温度为 20 的情况下匹配大型网络生成的软目标）来对小型网络进行正则化，其测试错误为 74 个。这表明，软目标可以将大量知识传递给蒸馏模型，包括从翻译后的训练数据中习得的泛化知识，即便迁移集不包含任何翻译内容。

当蒸馏网络的两个隐藏层各有 300 个或更多单元时，所有高于 8 的温度都产生了相当相似的结果。但当每层单元数大幅减少到 30 个时，2.5 到 4 范围内的温度效果明显优于更高或更低的温度。

然后，我们尝试从迁移集中剔除所有数字 3 的示例。因此，对于蒸馏模型而言，3 是一个它从未见过的神秘数字。尽管如此，蒸馏模型仅出现 206 次测试错误，其中 133 次错误出现在测试集中的 1010 个数字 3 上。大多数错误是由于针对数字 3 类别习得的偏置过低造成的。如果将该偏置增加 3.5（这会优化测试集上的整体性能），蒸馏模型的错误数会降至 109 次，其中仅有 14 次是针对数字 3 的。因此，凭借合适的偏置，尽管在训练过程中从未见过数字 3，蒸馏模型对测试集中数字 3 的识别正确率仍能达到 98.6%。如果迁移集仅包含训练集中的数字 7 和 8，蒸馏模型的测试错误率为 47.3%，但当将数字 7 和 8 的偏置降低 7.6 以优化测试性能时，错误率会降至 13.2%。

4 语音识别实验

在本节中，我们研究了集成用于自动语音识别（ASR）的深度学习（DNN）声学模型所产生的效果。我们表明，本文提出的蒸馏策略能够实现预期效果，即将模型集成蒸馏为单个模型，该单个模型的性能显著优于直接从相同训练数据中学习得到的同等规模模型。

最先进的自动语音识别（ASR）系统目前使用深度学习（DNNs），将从波形中提取的特征的（短）时间上下文映射到隐马尔可夫模型（HMM）离散状态上的概率分布 [4]。更具体地说，深度神经网络在每个时间点生成三音素状态簇上的概率分布，然后解码器会在隐马尔可夫模型的状态中找到一条路径，这条路径是使用高概率状态与生成在语言模型下具有较高概率的转录结果之间的最佳折中。

尽管可以（并且也希望）通过对所有可能路径进行边缘化处理，在训练深度学习（DNN）时将解码器（进而将语言模型）纳入考虑范围，但通常的做法是训练 DNN 进行逐帧分类，具体方式是（局部地）最小化网络所做预测与通过每个观测的真实状态序列强制对齐得到的标签之间的交叉熵：

$$\theta = \operatorname{argmax}_{\theta'} P(h_t | s_t; \theta')$$

其中， θ 是声学模型 P 的参数，该模型将时间 t 的声学观测 s_t 映射为“正确”的隐马尔可夫模型

（HMM）状态 h_t 的概率 $P(h_t | s_t; \theta')$ ，而这一状态是通过与正确的单词序列强制对齐确定的。该模型采用分布式随机梯度下降方法进行训练。

我们采用的架构包含 8 个隐藏层，每个隐藏层有 2560 个整流线性单元，以及一个包含 14000 个标签的最终 softmax 层（HMM 目标 h_t ）。输入为 26 帧，每帧包含 40 个梅尔刻度滤波器组系数，每帧间隔 10 毫秒，我们预测 21^{st} 帧的 HMM 状态。参数总数约为 8500 万。这是 Android 语音搜索所使用的声学模型的一个稍旧版本，可视为一个非常强大的基准模型。为了训练 DNN 声学模型，我们使用了约 2000 小时的英语口语数据，产生了约 7 亿个训练样本。该系统在我们的开发集上达到了 58.9% 的帧准确率和 10.9% 的词错误率（WER）。

System	Test Frame Accuracy	WER
Baseline	58.9%	10.9%
10xEnsemble	61.1%	10.7%
Distilled Single model	60.8%	10.7%

表 1: 帧分类准确率和词错误率 (WER) 表明, 经过蒸馏的单一模型的性能与用于生成软目标的 10 个模型的平均预测结果大致相当。

4.1 结果

我们训练了 10 个独立的模型来预测 $P(h_t|s_t; \theta)$, 使用的架构和训练流程与基线模型完全相同。这些模型通过不同的初始参数值进行随机初始化, 我们发现这会使训练后的模型产生足够的多样性, 从而让集成模型的平均预测结果显著优于单个模型。我们尝试通过改变每个模型所看到的数据集来增加模型的多样性, 但发现这并未显著改变结果, 因此我们选择了更简单的方法。在蒸馏过程中, 我们尝试了 [1, 2, 5, 10] 的温度值, 并对硬目标的交叉熵使用了 0.5 的相对权重, 其中粗体字表示用于表 1 的最佳值。

表 1 表明, 事实上, 我们的蒸馏方法确实能够从训练集中提取比单纯使用硬标签训练单个模型更多的有用信息。使用 10 个模型的集成所实现的帧分类准确率提升中, 超过 80% 被转移到了蒸馏模型上, 这与我们在 MNIST 上的初步实验中观察到的提升情况相似。由于目标函数的不匹配, 该集成在词错误率 (WER) 这一最终目标上的提升较小 (在 23K 词的测试集上), 但集成所实现的词错误率提升同样被转移到了蒸馏模型中。

我们最近注意到一项相关研究, 即通过匹配已训练好的更大模型的类别概率来学习小型声学模型 [8]。然而, 他们使用大型无标签数据集在温度为 1 的条件下进行蒸馏, 并且他们最好的蒸馏模型仅能将小型模型的错误率降低 28%, 这个降幅是大型模型和小型模型 (两者都使用硬标签训练时) 错误率差距的 28%。

5 在超大数据集上训练专家集成模型

训练模型集成是一种利用并行计算的非常简单的方法, 而通常认为集成在测试时需要过多计算的反对意见, 可以通过蒸馏来解决。然而, 集成还有另一个重要的问题: 如果各个模型都是大型神经网络, 且数据集非常庞大, 那么即使训练过程易于并行化, 训练时所需的计算量也会过大。

在本节中, 我们将给出这样一个数据集的示例, 并展示如何通过学习专注于类别中不同易混淆子集的专家模型, 来减少学习集成模型所需的总计算量。专注于进行细粒度区分的专家模型存在的主要问题是它们很容易过拟合, 我们将阐述如何通过使用软目标来防止这种过拟合。

5.1 JFT 数据集

JFT 是谷歌的一个内部数据集, 包含 1 亿张带有标签的图像, 涉及 15,000 个标签。在我们开展这项工作, 谷歌用于 JFT 的基准模型是一个深度卷积神经网络 [7], 该网络已在大量核心上通过异步随机梯度下降训练了约六个月。这种训练采用了两种并行方式 [2]。首先, 神经网络有许多副本在不同的核心集上运行, 处理训练集中不同的小批量数据。每个副本计算其当前小批量数据的平均梯度, 并将该梯度发送到分片参数服务器, 参数服务器则返回参数的新值。这些新值反映了自上次向副本发送参数以来, 参数服务器收到的所有梯度。其次, 每个副本通过将神经元的不同子集分配到每个核心上, 从而分布在多个核心上。集成训练是第三种并行方式, 可以对其进行封装。

JFT 1: 茶话会; 复活节; 新娘送礼会; 婴儿送礼会; 复活节兔子;JFT 2: 桥梁; 斜拉桥; 悬索桥; 高架桥; 烟囱;JFT 3: 丰田卡罗拉 E100; 欧宝 Signum; 欧宝雅特; 马自达 Familia;
--

表 2: 通过我们的协方差矩阵聚类算法计算得到的聚类中的示例类别

围绕另外两种类型, 但前提是有更多的核心可用。等待数年时间来训练一组模型并非可行之选, 因此我们需要一种更快的方法来改进基准模型。

5.2 专业模型

当类别数量非常庞大时, 让这个复杂的模型成为一个集成模型是合理的, 该集成模型包含一个在所有数据上训练的通用模型, 以及许多“专家”模型, 每个专家模型都在高度集中了某一极易混淆的类别子集(如不同种类的蘑菇)示例的数据上进行训练。通过将该专家模型不关注的所有类别合并为一个单一的“垃圾桶”类别, 可以大幅减小这类专家模型的 softmax 规模。

为了减少过拟合并分担学习较低层级特征检测器的任务, 每个专家模型都使用通用模型的权重进行初始化。然后, 通过训练专家模型对这些权重进行微调——训练时, 一半样本来自其专属子集, 另一半则从训练集的剩余部分中随机抽取。训练完成后, 我们可以通过将垃圾类别的对数几率增加专家类别过采样比例的对数, 来修正有偏的训练集。

5.3 为专家模型分配类别

为了给专家们推导出物体类别的分组, 我们决定聚焦于整个网络经常混淆的类别。尽管我们本可以计算混淆矩阵并将其用作找到此类聚类的方法, 但我们选择了一种更简单的方法, 该方法不需要真实标签来构建聚类。

特别是, 我们将一种聚类算法应用于我们通用模型预测结果的协方差矩阵, 这样一来, 一组经常被共同预测的类别 ζ^m 就会被用作我们其中一个专家模型 m 的目标。我们将 K-means 算法的在线版本应用于协方差矩阵的列, 得到了合理的聚类结果(如表 2 所示)。我们还尝试了几种其他聚类算法, 它们产生的结果相似。

5.4 使用专家集成进行推理

在研究专家模型被蒸馏时会发生什么之前, 我们先了解包含专家模型的集成模型表现如何。除了专家模型之外, 我们始终会配备一个通用模型, 这样我们就能处理那些没有对应专家模型的类别, 并且能够决定使用哪些专家模型。对于给定的输入图像 x , 我们通过两个步骤进行 top-one 分类:

步骤 1: 对于每个测试用例, 我们根据通用模型找到 n 个最可能的类别。将这组类别称为 k 。在我们的实验中, 我们使用了 $n = 1$ 。

步骤 2: 然后, 我们选取所有专家模型 m , 其易混淆类的特定子集 (ζ^m) 与 k 的交集非空, 并将其称为专家的活跃集 A_k 。(请注意, 该集合可能为空)。接着, 我们找到所有类别上的完整概率分布 q , 使其最小化:

$$KL(p^g \| q) + \sum_{m \in A_k} KL(p^m \| q) \quad (5)$$

其中, KL 表示 KL 散度, p^m 表示专家模型或通用全模型的概率分布。分布 p^m 是涵盖 m 的所有专家类别加上一个单一垃圾桶类别的分布, 因此, 在计算其与完整 q 分布的 KL 散度时, 我们将完整 q 分布分配给 m' 垃圾桶中所有类别的所有概率相加。

System	Conditional Test Accuracy	Test Accuracy
Baseline	43.1%	25.0%
+ 61 Specialist models	45.9%	26.1%

表 3: JFT 开发集上的分类准确率 (top 1)。

# of specialists covering	# of test examples	delta in top1 correct	relative accuracy change
0	350037	0	0.0%
1	141993	+1421	+3.4%
2	67161	+1572	+7.4%
3	38801	+1124	+8.8%
4	26298	+835	+10.5%
5	16474	+561	+11.1%
6	10682	+362	+11.3%
7	7376	+232	+12.8%
8	4703	+182	+13.6%
9	4706	+208	+16.6%
10 or more	9082	+324	+14.1%

表 4: 在 JFT 测试集上, 覆盖正确类别的专家模型数量对 Top 1 准确率的提升。

式 (5) 没有通用的闭式解, 但当所有模型为每个类别输出单一概率时, 其解要么是算术平均值, 要么是几何平均值, 这取决于我们使用的是 $KL(n, a)$ 还是 $KL(a, n)$ 。我们对 $a = \text{softmax}(z)$ 进行参数化 (使用 $\tau = 1$), 并采用梯度下降法, 针对式 (5) 优化 logits z 。注意, 这种优化必须针对每张图像进行。

5.5 结果

从训练好的基线完整网络出发, 专家模型的训练速度极快 (对于 JFT 数据集, 只需几天而非数周)。此外, 所有专家模型的训练完全独立进行。表 3 展示了基线系统以及基线系统与专家模型结合后的绝对测试准确率。使用 61 个专家模型时, 整体测试准确率相对提升了 4.4%。我们还报告了条件测试准确率, 即仅考虑属于专家类别的样本, 并将预测限制在该类别子集上的准确率。

在我们的 JFT 专家模型实验中, 我们训练了 61 个专家模型, 每个模型包含 300 个类别 (外加一个垃圾桶类别)。由于这些专家模型的类别集并非相互独立, 因此对于某个特定的图像类别, 我们往往有多个专家模型对其进行覆盖。表 4 展示了测试集样本数量、使用专家模型后在第 1 位正确的样本数量变化, 以及按覆盖某类别的专家模型数量划分的 JFT 数据集 top1 准确率的相对百分比提升。我们发现, 当有更多专家模型覆盖某个特定类别时, 准确率的提升更大, 这一总体趋势让我们备受鼓舞, 因为训练独立的专家模型非常易于并行化处理。

6 作为正则化器的软目标

我们关于使用软目标而非硬目标的一个主要观点是, 软目标中可以包含大量有用信息, 而这些信息不可能用单一的硬目标来编码。在本节中, 我们通过使用少得多的数据来拟合前面描述的基线语音模型的 8500 万个参数, 证明了这是一个非常显著的效果。表 5 显示, 仅使用 3% 的数据 (约 2000 万个样本) 时, 用硬目标训练基线模型会导致严重的过拟合 (我们进行了早停, 因为准确率在达到 44.5% 后急剧下降), 而使用软目标训练的同一模型几乎能够恢复完整训练集中的所有信息 (仅差约 2%)。更值得注意的是, 我们无需进行早停: 使用软目标的系统直接 “收敛” 到了 57%。这表明, 软目标是一种非常有效的方式, 能够将在所有数据上训练的模型所发现的规律传递给另一个模型。

System & training set	Train Frame Accuracy	Test Frame Accuracy
Baseline (100% of training set)	63.4%	58.9%
Baseline (3% of training set)	67.3%	44.5%
Soft Targets (3% of training set)	65.4%	57.0%

表 5: 软目标使新模型仅通过 3% 的训练集就能实现良好的泛化。这些软目标是通过在完整训练集上训练得到的。

6.1 使用软目标防止专家模型过拟合

在 JFT 数据集的实验中, 我们所使用的专家模型将所有非专业类别都归并到了一个单一的垃圾类别中。如果我们允许专家模型对所有类别进行完整的 softmax 计算, 或许会有一种比使用早停法更好的方式来防止它们过拟合。专家模型是在高度富集其专业类别的数据上进行训练的。这意味着其训练集的有效规模要小得多, 而且它很容易在自己的专业类别上发生过拟合。通过大幅缩小专家模型的规模无法解决这个问题, 因为那样的话, 我们就会失去通过对所有非专业类别进行建模所获得的非常有益的迁移效应。

我们使用 3% 的语音数据进行的实验有力地表明, 如果用通用模型的权重来初始化专业模型, 并且除了使用硬目标进行训练外, 再用非专业类别的软目标对其进行训练, 就能使专业模型几乎保留所有关于非专业类别的知识。这些软目标可以由通用模型提供。我们目前正在探索这种方法。

7 与专家混合体的关系

使用受过数据子集训练的专家, 这与专家混合模型 [6] 有些相似, 后者利用门控网络来计算将每个样本分配给每个专家的概率。在专家学习处理分配给它们的样本的同时, 门控网络也在学习根据专家对该样本的相对判别性能, 来选择将每个样本分配给哪些专家。利用专家的判别性能来确定习得的分配方式, 这比简单地对输入向量进行聚类并为每个聚类分配一个专家要好得多, 但这使得训练难以并行化: 首先, 每个专家的加权训练集以一种依赖于所有其他专家的方式不断变化; 其次, 门控网络需要比较不同专家在同一个样本上的表现, 才能知道如何修正其分配概率。这些困难意味着, 专家混合模型很少在它们可能最有益的场景中使用, 即那些拥有包含明显不同子集的海量数据集的任务。

并行化训练多个专家模型要容易得多。我们首先训练一个通用模型, 然后利用混淆矩阵来确定专家模型所要训练的子集。一旦这些子集确定下来, 专家模型就可以完全独立地进行训练。在测试时, 我们可以利用通用模型的预测结果来确定哪些专家模型是相关的, 并且只需要运行这些专家模型即可。

8 讨论

我们已经证明, 知识蒸馏在将知识从集成模型或大型高正则化模型转移到更小的蒸馏模型方面效果非常好。在 MNIST 数据集上, 即使用于训练蒸馏模型的迁移集缺少一个或多个类别的任何示例, 蒸馏仍然表现出显著的效果。对于一个深度声学模型 (该模型是 Android 语音搜索所使用模型的一个版本), 我们发现, 通过训练深度神经网络集成所获得的几乎所有改进, 都可以被蒸馏到一个相同大小的单一神经网络中, 而这种单一神经网络的部署要容易得多。

对于真正大型的神经网络来说, 训练一个完整的集成模型甚至可能是不可行的, 但我们已经证明, 通过学习大量的专家网络 (每个专家网络都学习区分高度易混淆聚类中的类别), 可以显著提升一个经过长时间训练的单个超大型网络的性能。不过, 我们尚未证明能够将这些专家网络中的知识提炼回那个单一的大型网络中。

致谢

我们感谢贾扬清在 ImageNet 上训练模型时提供的帮助，以及伊利亚·萨茨克弗和约拉姆·辛格的有益讨论。

参考文献

- [1] C. Bucilu[˘]a, R. Caruana 和 A. Niculescu-Mizil. 模型压缩。见《第 12 届 ACM SIGKDD 国际知识发现和数据挖掘会议论文集》(KDD '06)，第 535–541 页，美国纽约州纽约市，2006 年。ACM 出版社。
- [2] J. 迪恩、G. S. 科拉多、R. 蒙加、K. 陈、M. 德文、Q. V. 勒、M. Z. 毛、M. 兰扎托、A. 西尼尔、P. 塔克、K. 杨和 A. Y. 吴。大规模分布式深度网络。发表于《神经信息处理系统进展》，2012 年。
- [3] T. G. 迪特里希。机器学习中的集成方法。见《多分类器系统》，第 1-15 页。施普林格出版社，2000 年。
- [4] G. E. 辛顿、L. 邓、D. 余、G. E. 达尔、A. 穆罕默德、N. 贾特利、A. 西尼尔、V. 范霍克、P. 阮、T. N. 赛纳特及 B. 金斯伯里。用于语音识别中声学建模的深度神经网络：四个研究小组的共同观点。《IEEE 信号处理杂志》，29 (6)：82-97，2012。
- [5] G. E. 辛顿、N. 斯里瓦斯塔瓦、A. 克里热夫斯基、I. 苏茨克维尔和 R. R. 萨拉胡丁诺夫。通过防止特征检测器的协同适应改进神经网络。arXiv 预印本 arXiv:1207.0580，2012。
- [6] R. A. 雅各布斯、M. I. 乔丹、S. J. 诺兰和 G. E. 辛顿。局部专家的自适应混合。《神经计算》，3 (1)：79-87。1991。
- [7] A. 克里热夫斯基、I. 萨特斯克维尔与 G. E. 辛顿。《基于深度卷积神经网络的 ImageNet 图像分类》。收录于《神经信息处理系统进展》，第 1097–1105 页，2012 年。
- [8] 李 J、赵 R、黄 J、龚 Y。基于输出分布的标准学习小型深度神经网络。见《2014 年国际语音通信协会会议论文集》，第 1910-1914 页，2014 年。
- [9] N. Srivastava、G.E. Hinton、A. Krizhevsky、I. Sutskever 和 R. R. Salakhutdinov。Dropout：一种防止神经网络过拟合的简单方法。《机器学习研究期刊》，15 (1)：1929-1958，2014。